

Unit:11
DATABASE HANDLING

11.1 Server-side Database driven and tools: JDBC, ODBC, SQL:

JDBC:

- JDBC stands for Java Database Connectivity, which is a standard Java API for database-independent connectivity between the Java programming language and a wide range of databases.

The JDBC library includes APIs for each of the tasks mentioned below that are commonly associated with database usage.

- ✓ Making a connection to a database.
- ✓ Creating SQL or MySQL statements.
- ✓ Executing SQL or MySQL queries in the database.
- ✓ Viewing & Modifying the resulting records.
- All of these different executables are able to use a JDBC driver to access a database, and take advantage of the stored data.
- JDBC provides the same capabilities as ODBC, allowing Java programs to contain database-independent code.

JDBC Architecture:

- The JDBC API supports both two-tier and three-tier processing models for database access but in general, JDBC Architecture consists of two layers:
 - ✓ JDBC API: This provides the application-to-JDBC Manager connection.
 - ✓ JDBC Driver API: This supports the JDBC Manager-to-Driver Connection.
- The JDBC API uses a driver manager and database-specific drivers to provide transparent connectivity to heterogeneous databases.
- The JDBC driver manager ensures that the correct driver is used to access each data source.
- The driver manager is capable of supporting multiple concurrent drivers connected to multiple heterogeneous databases.

Following is the architectural diagram, which shows the location of the driver manager with respect to the JDBC drivers and the Java application –

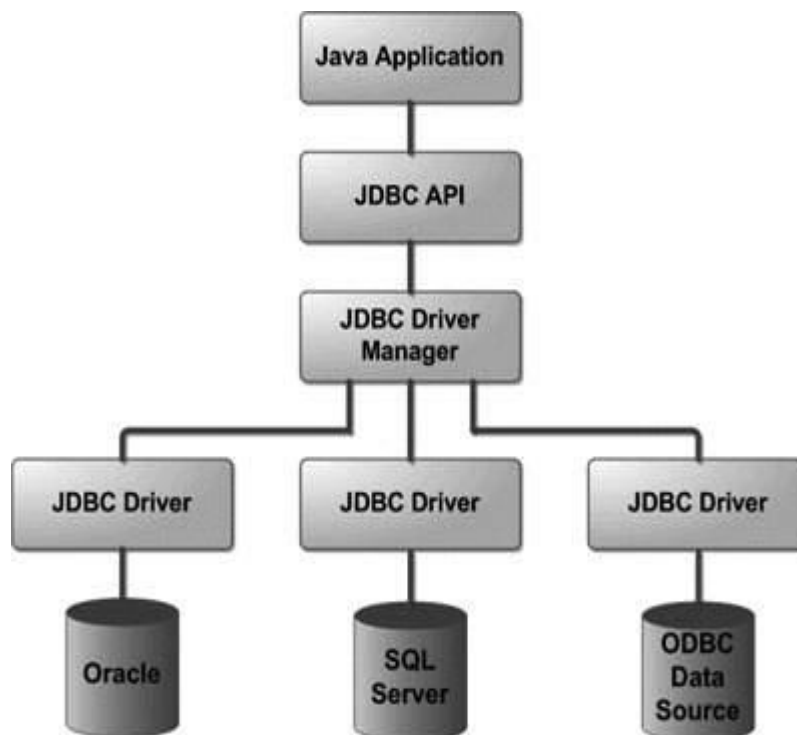


Fig: JDBC Architecture

Common JDBC Components:

The JDBC API provides the following interfaces and classes:

DriverManager: This class manages a list of database drivers. Matches connection requests from the java application with the proper database driver using communication sub protocol. The first driver that recognizes a certain subprotocol under JDBC will be used to establish a database Connection.

Driver: This interface handles the communications with the database server. You will interact directly with Driver objects very rarely. Instead, you use DriverManager objects, which manages objects of this type. It also abstracts the details associated with working with Driver objects.

Connection: This interface with all methods for contacting a database. The connection object represents communication context, i.e., all communication with database is through connection object only.

Statement: You use objects created from this interface to submit the SQL statements to the database. Some derived interfaces accept parameters in addition to executing stored procedures.

ResultSet: These objects hold data retrieved from a database after you execute an SQL query using Statement objects. It acts as an iterator to allow you to move through its data.

SQLException: This class handles any errors that occur in a database application

❖ JDBC Statements:[imp for exam]:

There are three types of JDBC statements available. Each of them is described below in detail.

1. The Statement Objects:

- It can be used for general-purpose access to the database.
- It is useful when you are using static SQL statements at runtime.
- Use this for general-purpose access to your database.
- The Statement interface cannot accept parameters.

Eg: `statement = connection.createStatement();`

- It has two part creating and closing statement.

Creating and Closing Statement Object:

- Before you can use a Statement object to execute a SQL statement, you need to create one using the Connection object's `createStatement()` method, as in the following example:

`Statement stmt = null;`

- A simple call to the `close()` method will do the job. If you close the Connection object first, it will close the Statement object as well. However, you should always explicitly close the Statement object to ensure proper clean-up.

Example creating and closing statement:

`Statement stmt = null;`

```
try {  
    stmt = conn.createStatement();  
    ...  
}  
catch (SQLException e) {  
    ...  
}  
finally {  
    stmt.close();  
}
```

2. The PreparedStatement Objects:

- Used for precompiling SQL statements that might contain input parameters.
- Use this when you plan to use the SQL statements many times.
- The PreparedStatement interface accepts input parameters at runtime.
- Improves performance: The performance of the application will be faster if you use PreparedStatement interface because query is compiled only once.

Eg:

`PreparedStatement pstmt = null;`

`String SQL = " SELECT authorID, firstName, lastName FROM authors "; pstmt = conn.prepareStatement(SQL);`

This statement gives you the flexibility of supplying arguments dynamically.

Creating and Closing PreparedStatement Object:

All parameters in JDBC are represented by the ? symbol, which is known as the parameter marker. You must supply values for every parameter before executing the SQL statement. A simple call to the close() method will do the job.

Example creating and closing statement:

```
PreparedStatement pstmt = null;
try {
    String SQL = "Update Employees SET age = ? WHERE id = ?";
    pstmt = conn.prepareStatement(SQL);
    ...
}
catch (SQLException e) {
    ...
}
finally {
    pstmt.close();
}
```

3. The CallableStatement Objects:

- Used to execute stored procedures that may contain both input and output parameters.
- Use this when you want to access the database stored procedures.
- The CallableStatement interface can also accept runtime input parameters.

Example:

CallableStatement

```
callableStatement=connection.prepareCall("{callcalculateStatistics(?,?)}");
```

Creating and Closing CallableStatement Object

- You must bind values to all the parameters before executing the statement, or you will receive an SQLException.
- A simple call to the close() method will do the job. If you close the Connection object first, it will close the CallableStatement object as well.
- However, you should always explicitly close the CallableStatement object to ensure proper clean-up.

Example creating and closing statement:

```
CallableStatement cstmt = null;
try {
    String SQL = "{call getEmpName (?, ?)}";
    cstmt = conn.prepareCall (SQL);
    ...
}
catch (SQLException e)
{
    ...
}
finally
{
    cstmt.close();
}
```

❖ JDBC Driver:[imp for Exam]:

- JDBC drivers implement the defined interfaces in the JDBC API, for interacting with your database server.
- For example, using JDBC drivers enable you to open database connections and to interact with it by sending SQL or database commands then receiving results with Java.

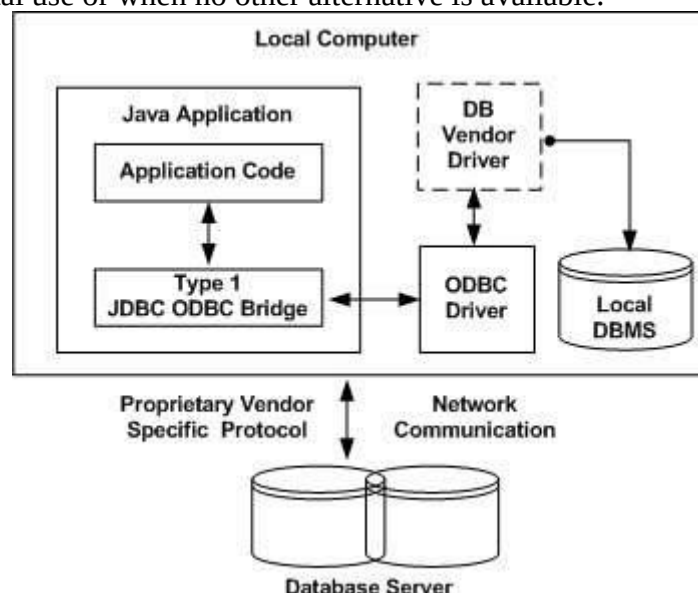
JDBC Drivers Types:

- JDBC driver implementations vary because of the wide variety of operating systems and hardware platforms in which Java operates.
- Sun has divided the implementation types into four categories.

Types 1, 2, 3, and 4, which is explained below –

Type 1: JDBC-ODBC Bridge Driver:

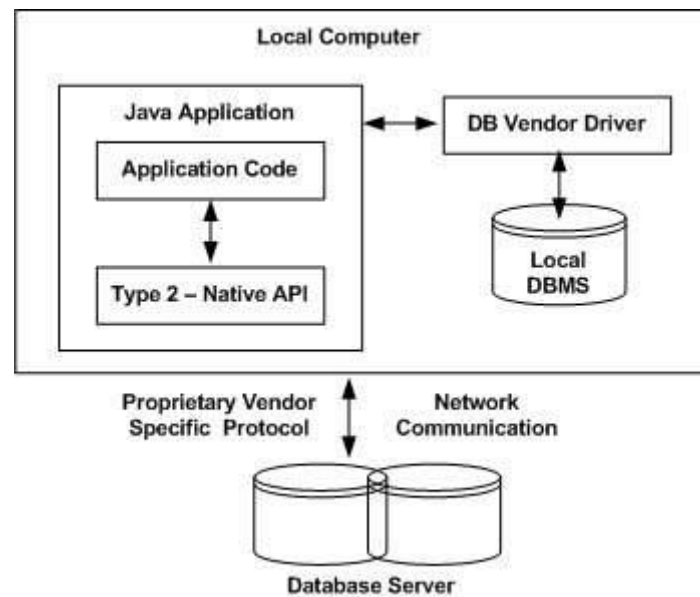
- In a Type 1 driver, a JDBC bridge is used to access ODBC drivers installed on each client machine.
- Using ODBC, requires configuring on your system a Data Source Name (DSN) that represents the target database.
- When Java first came out, this was a useful driver because most databases only supported ODBC access but now this type of driver is recommended only for experimental use or when no other alternative is available.



The JDBC-ODBC Bridge that comes with JDK 1.2 is a good example of this kind of driver.

Type 2: JDBC-Native API:

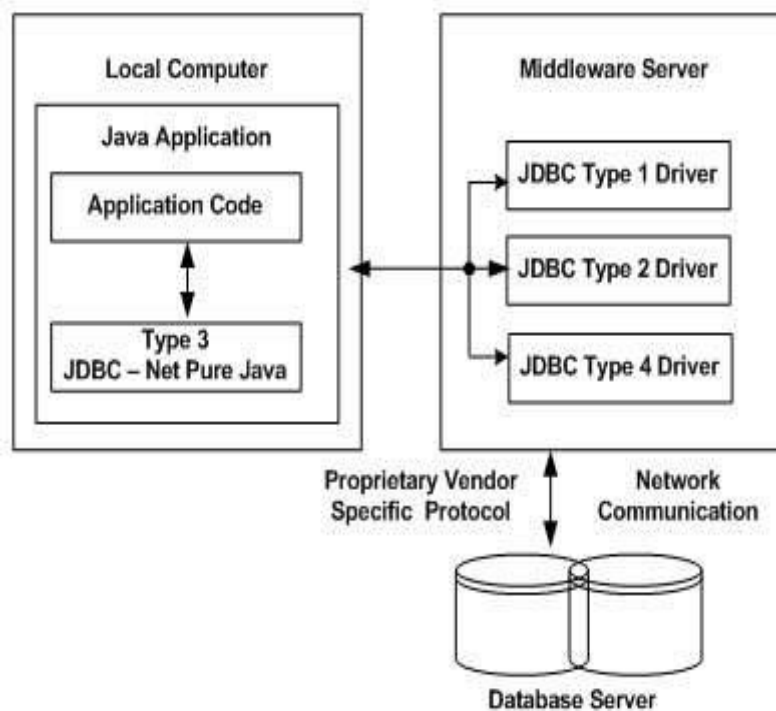
- In a Type 2 driver, JDBC API calls are converted into native C/C++ API calls, which are unique to the database.
- These drivers are typically provided by the database vendors and used in the same manner as the JDBC-ODBC Bridge.
- The vendor-specific driver must be installed on each client machine.
- If we change the Database, we have to change the native API, as it is specific to a database and they are mostly obsolete now, but you may realize some speed increase with a Type 2 driver, because it eliminates ODBC's overhead.



The Oracle Call Interface (OCI) driver is an example of a Type 2 driver.

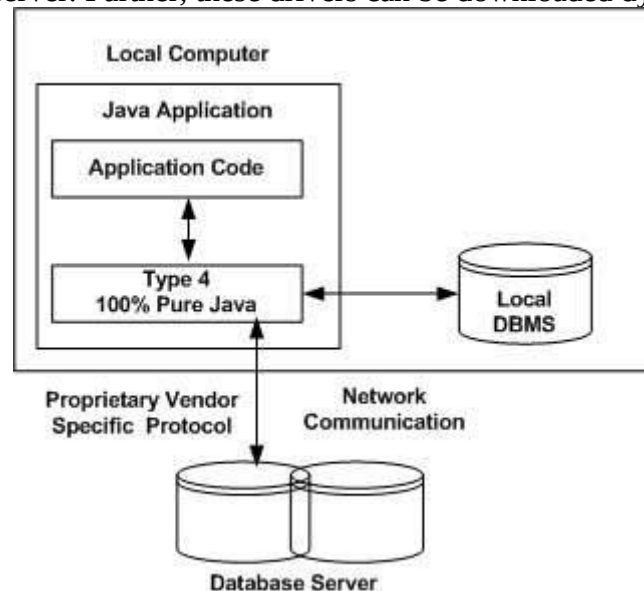
Type 3: JDBC-Net pure Java:

- In a Type 3 driver, a three-tier approach is used to access databases.
- The JDBC clients use standard network sockets to communicate with a middleware application server.
- The socket information is then translated by the middleware application server into the call format required by the DBMS, and forwarded to the database server.
- This kind of driver is extremely flexible, since it requires no code installed on the client and a single driver can actually provide access to multiple databases.



Type 4: 100% Pure Java:

- In a Type 4 driver, a pure Java-based driver communicates directly with the vendor's database through socket connection.
- This is the highest performance driver available for the database and is usually provided by the vendor itself.
- This kind of driver is extremely flexible, you don't need to install special software on the client or server. Further, these drivers can be downloaded dynamically.



MySQL's Connector/J driver is a Type 4 driver. Because of the proprietary nature of their network protocols, database vendors usually supply type 4 drivers.

❖ JDBC - Database Connections:

- After you've installed the appropriate driver, it is time to establish a database connection using JDBC.
- The programming involved to establish a JDBC connection is fairly simple.

Here are these simple four steps:

- ✓ Import JDBC Packages: Add import statements to your Java program to import required classes in your Java code.
- ✓ Register JDBC Driver: This step causes the JVM to load the desired driver implementation into memory so it can fulfill your JDBC requests.
- ✓ Database URL Formulation: This is to create a properly formatted address that points to the database to which you wish to connect.
- ✓ Create Connection Object: Finally, code a call to the *DriverManager* object's *getConnection()* method to establish actual database connection.

Import JDBC Packages:

- ✓ The Import statements tell the Java compiler where to find the classes you reference in your code and are placed at the very beginning of your source code.
- ✓ To use the standard JDBC package, which allows you to select, insert, update, and delete data in SQL tables, add the following imports to your source code:

```
import java.sql.* ; // for standard JDBC programs
import java.math.* ; // for BigDecimal and BigInteger support
```

Closing JDBC Connections:

- ✓ At the end of your JDBC program, it is required explicitly to close all the connections to the database to end each database session.
- ✓ A simple `close()` closes the connection with the method associated with connection object.
- ✓ To ensure that a connection is closed, provide a 'finally' block in your code.
- ✓ A *finally* block always executes, regardless of an exception occurs or not.

To close the above opened connection, you should call `close()` method as follows –
`conn.close();`

Connecting to the Database:

- ✓ `supposeconnection=DriverManager.getConnection(DATABASE_URL,"root","");`
- ✓ create a *Connection* object (package *java.sql*) referenced by *connection*.
- ✓ An object that implements interface *Connection* manages the connection between the *Java program and the database*.
- ✓ Connection objects enable programs to create SQL statements that manipulate databases.
- ✓ The program initializes connection with the result of a call to static method *getConnection* of class *DriverManager* (package *java.sql*), which attempts to connect to the database specified by its URL.
- ✓ Method *getConnection()* takes three arguments-a String that specifies the *database URL*, a String that specifies the *username* and a String that specifies the *password*.
- ✓ The URL locates the database (possibly on a network or in the local file system of the computer).
- ✓ The URL `jdbc:mysql://localhost:3306/books` specifies the protocol for communication (*jdbc*), the sub protocol for communication (*mysql*) and the location of the database (`//localhost:3306/books`, where *localhost* is the host running the MySQL server , 3306 is default *mysql* running port and *books* is the database name).
- ✓ The sub protocol *mysql* indicates that the program uses a MySQL-specific sub protocol to connect to the MySQL database.
- ✓ If the *DriverManager* cannot connect to the database, method *getConnection* throws a *SQLException* (package *java.sql*).
- ✓ Following Figure lists the JDBC driver names and database URL formats of several popular RDBMSs.

RDBMS	Database URL format
MySQL	<code>jdbc:mysql://hostname:portNumber/databaseName</code>
ORACLE	<code>jdbc:oracle:thin:@hostname:portNumber:databaseName</code>
DB2	<code>jdbc:db2:hostname:portNumber/databaseName</code>
PostgreSQL	<code>jdbc:postgresql://hostname:portNumber/databaseName</code>
Java DB/Apache Derby	<code>jdbc:derby:databaseName (embedded)</code> <code>jdbc:derby://hostname:portNumber/databaseName (network)</code>
Microsoft SQL Server	<code>jdbc:sqlserver://hostname:portNumber;databaseName=databaseName</code>
Sybase	<code>jdbc:sybase:Tds:hostname:portNumber/databaseName</code>

Important Questions from this unit

1. Explain about types of drivers in jdbc?
2. Explain the types of statement?
3. Differentiate between JDBC and ODBC?
4. How do you connect java with database explain?

Differences Between odbc and jdbc:

	ODBC	JDBC
Definition	ODBC - Open Database Connectivity.	JDBC - Java Database Connectivity.
Languages	Can be used for C/C++/java.	Only for Java.
Platform	Windows	Most of the platforms being cross-platform compatibility.
Driver Development	ODBC drivers are implemented in native languages like C/C++.	JDBC drivers are implemented in Java.
Performance	ODBC drivers are faster.	JDBC drivers are slower than ODBC drivers.
Type	ODBC is procedural.	JDBC is object oriented.
Code	Difficult to understand	Easy to understand

ALL THE BEST FOR YOUR EXAMS